08_pv 스크립트는 Synopsys의 **IC Compiler II (ICC2)**와 물리 검증 툴인 **IC Validator (ICV)**를 연동하는 **In-Design PV (**물리 검증 및 메탈 필 삽입**)** 단계의 전체 Tcl 스크립트입니다. [2026]

앞선 `chip_finish` 단계의 결과물을 가져와, 외부 검증 툴로 완전히 빠져나가기 전에 ICC2 내부에서 **1)** 디자인 룰**(DRC)**을 체크 및 자동 수정하고, **2)** 칩 전체의 메탈 밀도를 맞추기 위한 메탈 필**(Metal Fill)**을 삽입하는 공정입니다.

이번에도 기능 블록별로 나누어 한 줄도 빠짐없이 상세하게 해석해 드리겠습니다.

---

1. 초기 환경 설정 및 외부 ICV 툴 모듈 로드

tcl

```
#made by kjinsoo
# module load
#module load icc2/2021.06-SP5
#module load icv/2022.12-SP5
# open icc2_shell
#icc2_shell -gui | tee -i ./10_pv.log

#set current step & before step
set step DRC
set before_step chip_finish
#source design_setup.tcl
source -e ./icc2_common_env/design_setup.tcl

#set_host_options -max_cores 16

#open lib
open_lib ../07_chip_finish/nlib/ORCA_${before_step}.nlib
#copy lib
copy_lib -from ../07_chip_finish/nlib/ORCA_${before_step}.nlib -to
./nlib/ORCA_${step}.nlib
#close & open current lib
close_lib -all
open_lib ./nlib/ORCA_${step}.nlib
#open block
open_block ./nlib/ORCA_${step}.nlib:ORCA.design
#link_block
link_block -rebind
```

코드를 사용할 때는 주의가 필요합니다.

- 해설: 물리 검증 단계 실행 로그(`10_pv.log`) 및 단계 변수를 선언합니다. 주석에 명시된 대로 이 단계는 정밀 물리 검증을 위해 **IC Validator (icv)** 모듈을 함께 로드해야 합니다.
- 앞선 `chip_finish` 결과 라이브러리를 카피하여 `DRC` 전용 독립 라이브러리로 오픈 및 동기화(`link_block -rebind`)합니다.

---

2. Sign-off DRC 검증을 위한 사전 옵션(App Options) 튜닝

tcl

```
####################
# ICV In-Design DRC
####################
#set app options before run DRC

set_app_options -list {signoff.check_drc.runset
{./runset/saed32nm_1p9m_drc_rules.rs}}
set_app_options -name signoff.check_drc.max_errors_per_rule -value 1000
set_app_options -name signoff.check_drc.run_dir -value "z_icvDRC_run"
set_app_options -list {signoff.check_drc.read_design_views {*}}
```

코드를 사용할 때는 주의가 필요합니다.

- **`check_drc.runset`**: 파운드리가 제공한 32nm 공정용 **DRC** 룰셋 파일(**`saed32nm_1p9m_drc_rules.rs`**)의 경로를 지정합니다. ICV 툴은 이 규칙을 기반으로 이격 거리, 두께 위반 등을 잡아냅니다.
- **`max_errors_per_rule -value 1000`**: 하나의 디자인 룰 항목당 데이터베이스에 저장할 최대 에러 개수를 1000개로 제한하여, 특정 룰의 대량 에러로 인해 로그나 메모리가 마비되는 것을 방지합니다.
- **`run_dir -value "z_icvDRC_run"`**: ICV 검증 엔진이 구동되면서 생성할 임시 파일 및 에러 결과물이 저장될 작업 디렉토리 명을 지정합니다.
- **`read_design_views {*}`**: 하위 셀 인스턴스의 물리 모양을 단순 추상화 뷰(`FRAME`)가 아닌, 실제 정밀 배선 정보가 담긴 디자인 뷰(**`design`**)로 싹 다 읽어 들여 **wild card(`*`)** 기준으로 초정밀 검증을 수행하도록 지시합니다.

---

3. Sign-off DRC 실행 및 위반 항목 자동 수정

tcl

```
#Run DRC
signoff_check_drc -check_all_runset_layers true

#Fix DRC Violation after Run DRC
```

```
#set app option
set_app_options -list {signoff.fix_drc.init_drc_error_db
"z_icvDRC_run"}

#fix drc
signoff_fix_drc
```

코드를 사용할 때는 주의가 필요합니다.

- **`signoff_check_drc ...`:** 설정된 룰셋을 기반으로 백라운드에서 ICV 툴을 실행하여 칩 전체 레이어에 대한 정밀 DRC 검사를 수행합니다.
- **`fix_drc.init_drc_error_db`:** 방금 생성된 DRC 에러 데이터베이스(`z_icvDRC_run`)의 위치를 지정하여 수정 엔진이 이를 참조하게 만듭니다.
- **`signoff_fix_drc`:** 발견된 DRC 위반 지점(배선 쇼트, 스페이싱 위반 등)을 P&R 엔진의 라우터 기능을 이용해 물리적으로 알아서 우회 배선하거나 수정**(Auto-Fixing)**합니다. 외부 검증 툴로 나가기 전 에러를 대폭 줄여주는 강력한 In-Design 기능입니다.

---

4. 메탈 필(Metal Fill) 삽입을 위한 사전 옵션 설정

tcl

```
###################
# run Metal Fill
###################
#set app_option for Metal Fill
set_app_options -list {signoff.create_metal_fill.runset
{./runset/saed32nm_1p9m_mfill_rules.rs}}
set_app_options -list {signoff.create_metal_fill.read_design_views {*}}
```

코드를 사용할 때는 주의가 필요합니다.

- **`create_metal_fill.runset`:** 반도체 제조 시 특정 영역에 메탈이 너무 없거나 많으면 화학적 기계적 연마(CMP) 공정에서 웨이퍼가 편평하게 깎이지 않습니다. 이를 막기 위해 가짜 메탈 조각을 채워 넣는 메탈 필 전용 룰셋 파일**(`saed32nm_1p9m_mfill_rules.rs`)**의 경로를 지정합니다.
- 마찬가지로 모든 하위 셀의 내부 레이아웃 구조(`design` 뷰)를 완벽히 고려하여 메탈 필이 중첩되지 않도록 설정합니다.

---

5. 타이밍 마진을 고려한 메탈 필 생성 및 ECO 플로우(주석)

tcl

```
#create Metal fill
signoff_create_metal_fill \
      -timing_preserve_setup_slack_threshold 0.05

#Timing analysis shows new small violations
#remvoe Filler Cells, re-run roue_opt, then re-insert Filler Cells

#Run auto-eco Metal Fill
#signoff_create_metal_fill \
      -auto_eco true \
      -timing_preserve_setup_slack_threshold 0.05
```

코드를 사용할 때는 주의가 필요합니다.

- **`signoff_create_metal_fill`:** 레이아웃의 빈 공간에 격자 형태로 메탈 필(Dummy Metal)을 자동으로 대량 생성합니다.
- **`-timing_preserve_setup_slack_threshold 0.05`:** 메탈 조각이 실제 신호선(Signal) 주변에 너무 바짝 붙으면 기생 정전용량(Capacitance)이 증가해 타이밍 지연이 나빠집니다. 따라서 **Setup** 타이밍 여유**(Slack)**가 최소 **0.05ns(50ps)** 이상 남아있는 안전한 영역에만 메탈 필을 깔도록 제한하여 타이밍 붕괴를 예방합니다.
- 주석 내용 해설 **(`-auto_eco true`)**: 메탈 필 삽입 후 미세한 타이밍 위반이 발생할 경우를 대비한 플로우 가이드입니다. 주석 처리된 이 옵션은 기존 메탈 필 중 타이밍에 악영향을 준 개체만 똑똑하게 골라내어 지우고 재생성하는 **Auto-ECO** 메탈 필 메커니즘을 의미합니다.

---

6. 최종 데이터베이스 마감 및 종료

tcl

```
#save & exit
save_lib -all
exit
```

코드를 사용할 때는 주의가 필요합니다.

- 해설**:** ICV 연동을 통한 DRC 수정 및 CMP 보정용 메탈 필 삽입이 완료된 현재의 최종 물리 검증용 라이브러리를 안전하게 저장(`save_lib -all`)하고 ICC2 쉘을 최종 종료(`exit`)합니다.
- 이 단계가 성공적으로 끝나면 칩은 완벽한 물리적 결함 제로 상태가 되어, 가상 검증이 아닌 공장의 마스크 제작용 진짜 **Sign-off PV (Calibre, ICV** 단독 구동**)** 단계로 즉시 진입할 수 있게 됩니다.

💡 추가 디버깅 가이드 (In-Design 팁)

- 만약 `signoff_check_drc` 수행 후 `signoff_fix_drc` 단계를 거쳤음에도 불구하고 기본 메탈 레이어**(M1~M3 등)**에서 대량의 **DRC** 위반이 남는다면, 앞선 `route_auto` 단계에서 라우팅 트랙 혼잡도(Congestion)가 풀리지 않은 채 넘어왔을 확률이 높습니다.
- 메탈 필 삽입 후 **`report_timing`** 결과 슬랙**(Slack)**이 급격히 저하된다면 `-timing_preserve_setup_slack_threshold` 값을 `0.08` 또는 `0.1` 등으로 더 보수적으로 높여 메탈 필과 신호선 간의 이격 거리를 더 확보하는 것이 좋습니다.